2025

Data Engineering Project- Airline on-time / delayed performance analysis

CIS 9760- BIG DATA TECHNOLOGIES

EVELYN NYASEMBI

| evelyn.nyasembi@baruchmail.cuny.edu

Table of Contents

Milestone 1 ..... *2*

Proposal **2**
- **Dataset Description** 2
- **Data Source** 2
- **Dataset Attributes** 2
- **Prediction Objective** 3
- **Machine Learning Approach** 3

Milestone 2 ..... *3*

Data Acquisition **3**
- **Summary** 3

Milestone 3 ..... *4*

EDA and Data Cleaning **4**
- **Summary and Issues predicted for feature engineering** 4
- **Steps in milestone 3 for EDA** 4
- **Viz. 1** 6
- **Viz. 2** 7
- **Viz. 3** 7
- **Viz. 4** 8
- **Viz. 5** 8
- **Viz. 6** 9
- **Steps in milestone 3 for data cleaning** 10

Milestone 4 ..... *11*

Feature Engineering and Modeling **11**
- **Description** 11
- **Flight Features treatment** 12
- **Steps in milestone** 13

Milestone 5 ..... *13*

Visualization **13**
- **Summary** 13
- **Viz. 7** 14
- **Viz. 8** 14
- **Viz. 9** 15
- **Viz. 10** 16
- **Viz. 11** 16

Appendix A **17**

Data Acquisition Script **17**
- **Code** 17

Milestone 6 ..... *19*

Summary and Conclusions **19**

Appendix B **20**

EDA **20**
- **Code** 20

Appendix C **22**

Data cleaning **22**
- **Code** 22

Appendix D **27**

Feature engineering and modeling **27**
- **Code** 27

Appendix e **32**

Visualization **32**
- **Code** 32

Citations and Tools used **34**

Milestone 1

Proposal

Dataset Description

The dataset contains records of airline performance, including details on flight schedules, delays, cancellations, and other operational metrics. This dataset can be leveraged to analyze trends in airline efficiency, punctuality, and factors influencing delays and cancellations.

Data Source

The dataset is available for download at the following location: Airline on-time performance analysis

Dataset Attributes

The dataset includes 29 columns that capture relevant flight details. Attributes are:

Year	1987-2008
Month	1-12
DayofMonth	1-31
DayofWeek	1(Monday) – 7(Sunday)
DepTime	actual depature time (local, hhmm)
CRSDepTime	scheduled departure time (local, hhmm)
ArrTime	actual arrival time (local, hhmm)
CRSArrTime	scheduled arrival time (local,hhmm)
UniqueCarrier	unique carrier code
Flight Number	flight number
Tailnum	plane tail number
ActualElapsedTime	in minutes
CRSElapsedTime	in minutes
AirTime	in minutes
ArrDelay	arrival delay, in minutes
DepDelay	departure delay, in minutes
Origin	origin IATA airport code
Dest	destination IATA airport code
Distance	in miles
TaxiIn	tax in time, in minutes
TaxiOut	taxi out time in minutes
Cancelled	was the flight cancelled
CancellationCode	reason for cancellation (A = carrier, B = weather, C = NAS, D = security)
Diverted	1 = yes, 0 = no
CarrierDelay	in minutes
WeatherDelay	in minutes
NasDelay	in minutes
SecurityDelay	in minutes
LateAircraftDelay	in minutes

Prediction Objective

The goal of this analysis is to predict flight delays using machine learning techniques. Based on the nature of the data and the objectives, **logistic regression** will be used to classify flights as "On Time" or "Delayed" based on whether the arrival delay exceeds a defined threshold. This approach allows for flexible classification of delays and helps identify patterns that contribute to late arrivals.

Machine Learning Approach

By implementing logistic regression, this project aims to classify flights as "Delayed" or "On Time" based on whether the arrival delay exceeds a defined threshold. We will experiment with different thresholds (e.g., 14, 30, and 50 minutes) to assess their impact on classification accuracy. Other models, such as decision trees or random forests, may also be explored to improve performance.

In conclusion, this analysis will provide valuable insights into flight delays, enabling airlines and passengers to make more informed travel decisions. By identifying key contributing factors, the findings can support the development of better strategies to minimize disruptions in air travel.

Milestone 2

Data Acquisition

Summary

The data acquisition and storage process involved collecting, storing, and organizing data for further processing. The first step was fetching the dataset from **Kaggle** using a shell script with the curl command. The dataset archive (airline-data.zip) was downloaded directly from the Kaggle repository. After downloading, the dataset was extracted using the unzip command. The extracted files included airline.csv.shuffle and carriers.csv, which were required for analysis.

A **Google Cloud Storage (GCS) bucket** named my-bigdata-project-en was created to store the dataset. The bucket was configured with **standard storage class** and **uniform bucket-level access** to manage permissions and access. Authentication was completed using the gcloud auth login command, allowing interaction with the storage bucket. The extracted dataset files were uploaded to the /landing folder in the GCS bucket using the gcloud storage cp command. A verification step was performed to confirm that the files were successfully transferred to the GCS bucket. The contents of the bucket were listed using gcloud commands.

Additional empty folders were created in the GCS bucket for data processing. These folders included **cleaned**, **trusted**, **code**, and **models**. Empty placeholder files were uploaded to these folders using the gcloud storage cp /dev/null command. These steps established a structured process for data acquisition and storage, ensuring the availability of raw data for further analysis. The structured storage allowed for efficient data processing and integration into subsequent workflows.

Milestone 3

EDA and Data Cleaning

Summary and Issues predicted for feature engineering

The first step in the data analysis project focused on cleaning the dataset to ensure it was suitable for analysis and machine learning modeling. The dataset included critical flight information such as arrival and departure delays, flight distances, and airport codes. Invalid values were removed by filtering out rows where ActualElapsedTime, DepDelay, and ArrDelay were zero or negative, as such values are not meaningful in the context of flight delays.

Missing values were addressed by dropping rows with null entries in key columns including ArrDelay, DepDelay, Cancelled, Distance, Month, Year, and DayOfWeek. This ensured the analysis was based on complete and reliable data. Irrelevant columns such as AirTime, ArrTime, and DepTime were also dropped to simplify the dataset and reduce noise in the modeling process.Data types were standardized by casting columns to appropriate formats. For example, ArrDelay and DepDelay were cast to float to support calculations such as delay aggregation and averages.

Despite the successful cleaning process, the next phase—feature engineering—presents several challenges. Time-based features like Month, DayOfWeek, and Year can significantly influence delay patterns, and selecting the right encoding strategies will be essential to capture seasonal or temporal trends. Categorical variables such as Origin, Dest, and FlightNum also require appropriate encoding (e.g., one-hot or label encoding) to be effectively used in machine learning models.

Another challenge is class imbalance, as the dataset may contain disproportionately more on-time flights than delayed ones. This imbalance can bias model predictions, making it necessary to consider mitigation strategies such as resampling or adjusting class weights.

In summary, while the dataset has been successfully cleaned, careful handling of time-based and categorical features, along with strategies to manage class imbalance, will be critical to building an accurate and robust flight delay prediction model.

Steps in milestone 3 for EDA

1. Using a 5 cluster dataproc node

```
gcloud dataproc clusters create cluster-a8fe --enable-component-gateway
--region us-central1 --master-machine-type n2-standard-4 --master-boot-
disk-type pd-balanced --master-boot-disk-size 100 --num-workers 2 --
worker-machine-type n2-standard-4 --worker-boot-disk-type pd-balanced -
-worker-boot-disk-size 100 --image-version 2.2-debian12 --optional-
components JUPYTER --max-idle 3600s –project ProjectID
```

2. Using PySpark, MatplotLib and Seaborn
   a. Getting data

[6]:
```
# Set the logging level to only show errors
sc.setLogLevel("ERROR")
```

[7]:
```
# Set up the path to a file
bucket = 'gs://my-bigdata-project-en'
landing_folder = f"{bucket}/landing/"
cleaned_folder = f"{bucket}/cleaned/"
```

[9]:
```
# Create a Spark Dataframe from some Amazon reviews data
sdf = spark.read.csv(landing_folder, sep=',', header=True, inferSchema=True)
```

b. Schema

```
# Check the schema
sdf.printSchema()
```

```
root
 |-- ActualElapsedTime: string (nullable = true)
 |-- AirTime: string (nullable = true)
 |-- ArrDelay: string (nullable = true)
 |-- ArrTime: string (nullable = true)
 |-- CRSArrTime: integer (nullable = true)
 |-- CRSDepTime: integer (nullable = true)
 |-- CRSElapsedTime: string (nullable = true)
 |-- CancellationCode: string (nullable = true)
 |-- Cancelled: integer (nullable = true)
 |-- CarrierDelay: string (nullable = true)
 |-- DayOfWeek: integer (nullable = true)
 |-- DayofMonth: integer (nullable = true)
 |-- DepDelay: string (nullable = true)
 |-- DepTime: string (nullable = true)
 |-- Dest: string (nullable = true)
 |-- Distance: string (nullable = true)
 |-- Diverted: integer (nullable = true)
 |-- FlightNum: integer (nullable = true)
 |-- LateAircraftDelay: string (nullable = true)
 |-- Month: integer (nullable = true)
 |-- NASDelay: string (nullable = true)
 |-- Origin: string (nullable = true)
 |-- SecurityDelay: string (nullable = true)
 |-- TailNum: string (nullable = true)
 |-- TaxiIn: string (nullable = true)
 |-- TaxiOut: string (nullable = true)
 |-- UniqueCarrier: string (nullable = true)
 |-- WeatherDelay: string (nullable = true)
 |-- Year: integer (nullable = true)
```

c. Number of records

```
# Display the number of records
record_count = sdf.count()
print(f'Total number of records: {record_count}')
```

```
[Stage 2:============================================================>(89 + 1) / 90]
Total number of records: 123536460
```

d. List of variables

```
# Display the list of columns
columns = sdf.columns
print(f'List of Variables: {columns}')
```

```
List of Variables: ['ActualElapsedTime', 'AirTime', 'ArrDelay', 'ArrTime', 'CRSArrTime', 'CRSDepTime', 'CRSElapsedTime', 'CancellationCode', 'Cancelled', 'Car
rierDelay', 'DayOfWeek', 'DayofMonth', 'DepDelay', 'DepTime', 'Dest', 'Distance', 'Diverted', 'FlightNum', 'LateAircraftDelay', 'Month', 'NASDelay', 'Origin',
'SecurityDelay', 'TailNum', 'TaxiIn', 'TaxiOut', 'UniqueCarrier', 'WeatherDelay', 'Year']
```

e. Filter the data and check count

[9]:
```
# Filter out rows where departure or arrival delay is 0 or negative
sdf = sdf.filter(sdf['ActualElapsedTime'] > 0)
sdf = sdf.filter(sdf['DepDelay'] > 0)
sdf = sdf.filter(sdf['ArrDelay'] > 0)
```

[19]:
```
# Get the new number of records after dropping
print("After filtering and droping missing values")
sdf.count()
```

```
After filtering and droping missing values
```

[19]:
```
37588884
```

f. Count of Missing values

```
]: # Count missing values per column
na_counts = sdf.select(
    [(F.count(F.when(F.col(c).isNull() | (F.col(c) == 'NA'), c)).alias(c)) for c in sdf.columns]
)

# Display the result
na_counts.show()
```

```
[Stage 23:=============================================================>(89 + 1) / 90]
+----------------+-------+--------+--------+----------+-----------+----------------+-----------------+---------+-----------+---------+----------+--------+-----------+--------+-----------+-------+
|ActualElapsedTime|AirTime|ArrDelay|ArrTime|CRSArrTime|CRSDepTime|CRSElapsedTime|CancellationCode|Cancelled|CarrierDelay|DayOfWeek|DayofMonth|DepDelay|DepTime|Dest|Distance|Diverted|FlightNum|LateAircraftDelay|Month|NASDelay|Origin|SecurityDelay|TailNum|TaxiIn|TaxiOut|UniqueCarrier|WeatherDelay|Year|
+----------------+-------+--------+--------+----------+-----------+----------------+-----------------+---------+-----------+---------+----------+--------+-----------+--------+-----------+-------+
|         2587530|39266398| 2589020|2585969|      1491|       1491|           27725|       122801754|     1491|    89330924|     1491|      1491| 2303627|2303627|1491|  203491|    1491|     1491|         89330924| 1491|89330924|  1491|     89330924|37386911|37398786|37384195|         1491|    89330924|1491|
+----------------+-------+--------+--------+----------+-----------+----------------+-----------------+---------+-----------+---------+----------+--------+-----------+--------+-----------+-------+
```

g. Min/Max Dates

```
[11]: # min and max dates.
sdf.select("DayOfWeek", "DayofMonth", "Month","Year").summary("count", "min", "max").show()
```

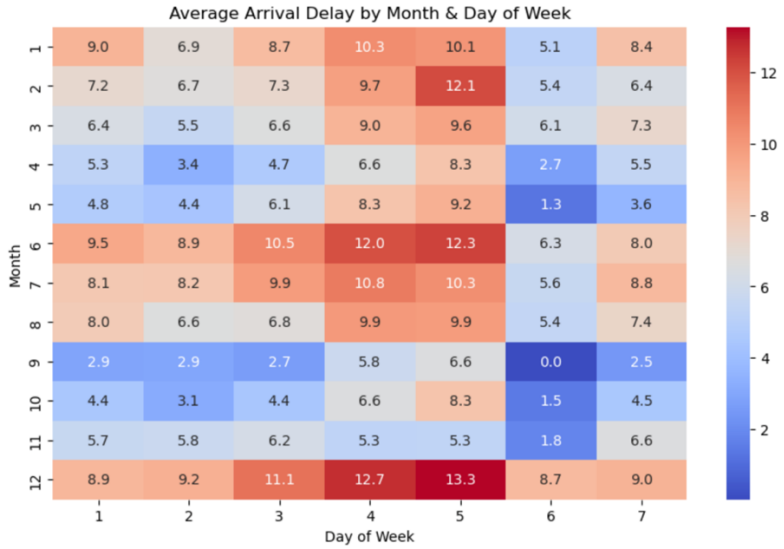
```
[Stage 5:=============================================================>(89 + 1) / 90]
+-------+---------+----------+--------+--------+
|summary|DayOfWeek|DayofMonth|   Month|    Year|
+-------+---------+----------+--------+--------+
|  count| 37660708|  37660708|37660708|37660708|
|    min|        1|         1|       1|    1987|
|    max|        7|        31|      12|    2008|
+-------+---------+----------+--------+--------+
```

h. graphs / charts to show the distribution of data

Viz. 1

```
: delay_heatmap = sdf.groupBy("Month", "DayOfWeek") \
                     .avg("ArrDelay") \
                     .toPandas().pivot(index="Month", columns="DayOfWeek", values="avg(ArrDelay)")

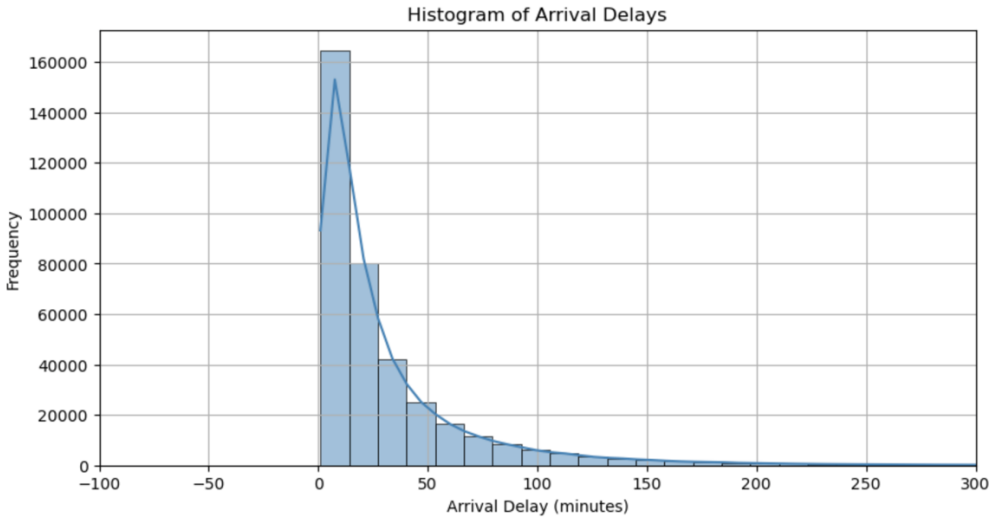
plt.figure(figsize=(10, 6))
sns.heatmap(delay_heatmap, annot=True, fmt=".1f", cmap="coolwarm")
plt.title("Average Arrival Delay by Month & Day of Week")
plt.xlabel("Day of Week")
plt.ylabel("Month")
plt.show()
```



Viz. 2

[20]:
```
# Sample and convert to pandas to keep it fast and memory-safe
pandas_df = sdf.select("ArrDelay").dropna().sample(False, 0.01).toPandas()

# Plot histogram
plt.figure(figsize=(10, 5))
sns.histplot(pandas_df["ArrDelay"], bins=100, kde=True, color="steelblue")
plt.title("Histogram of Arrival Delays")
plt.xlabel("Arrival Delay (minutes)")
plt.ylabel("Frequency")
plt.xlim(-100, 300)  # Optional: zoom into the most common delay range
plt.grid(True)
plt.show()
```



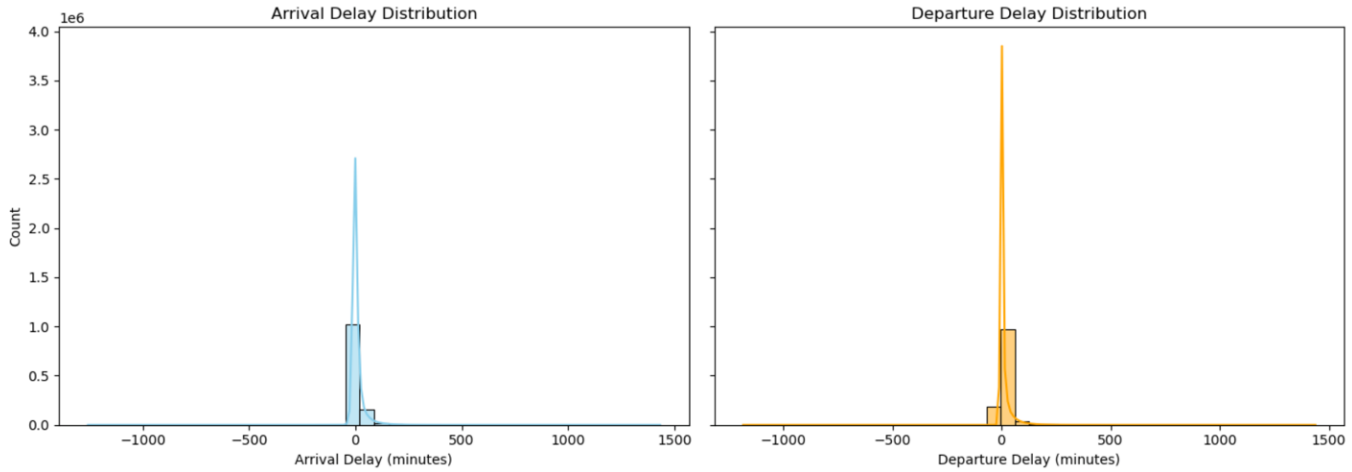
Viz. 3

```
fig, axs = plt.subplots(1, 2, figsize=(14, 5), sharey=True)

sns.histplot(pandas_df["ArrDelay"], kde=True, bins=40, color="skyblue", ax=axs[0])
axs[0].set_title("Arrival Delay Distribution")
axs[0].set_xlabel("Arrival Delay (minutes)")

sns.histplot(pandas_df["DepDelay"], kde=True, bins=40, color="orange", ax=axs[1])
axs[1].set_title("Departure Delay Distribution")
axs[1].set_xlabel("Departure Delay (minutes)")

plt.tight_layout()
plt.show()
```

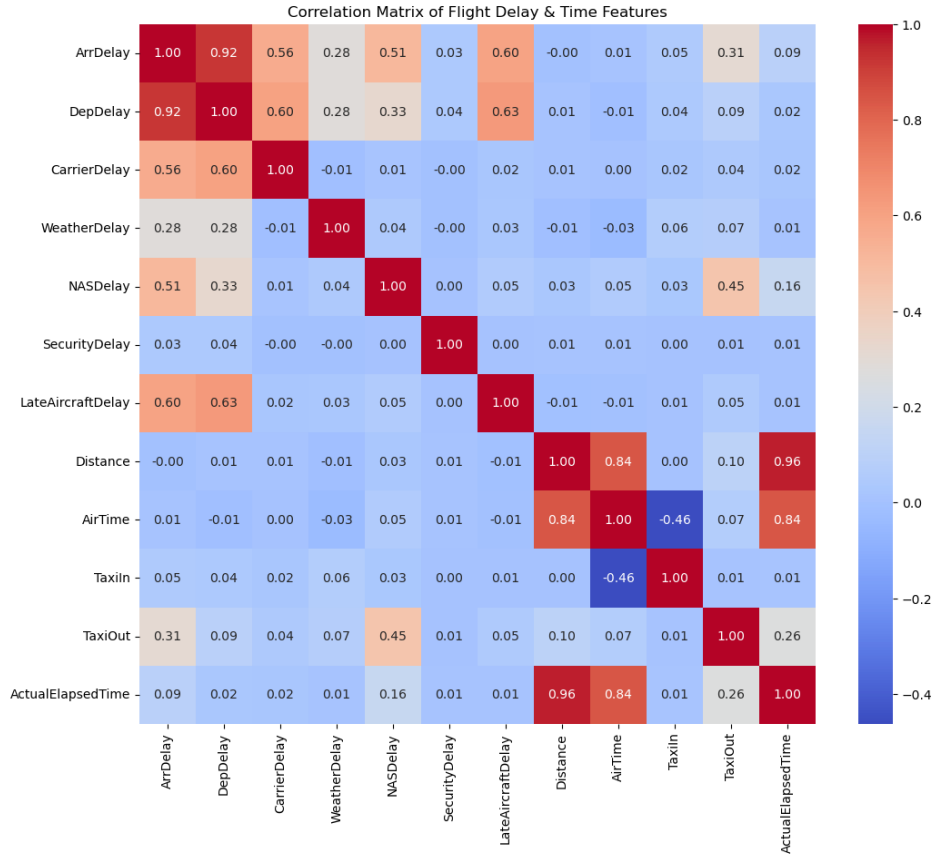


Viz. 4

```
# Correlation Heatmap (for numeric relationships) Strong positive correlation (close to 1) → variables rise together
#Strong negative correlation (close to −1) → one rises, the other falls
#Near 0 → weak or no linear relationship
cols = [
    "ArrDelay", "DepDelay", "CarrierDelay", "WeatherDelay",
    "NASDelay", "SecurityDelay", "LateAircraftDelay",
    "Distance", "AirTime", "TaxiIn", "TaxiOut", "ActualElapsedTime"
]

# Sample and convert to pandas
numeric_df = sdf.select(cols).dropna().sample(False, 0.01).toPandas()

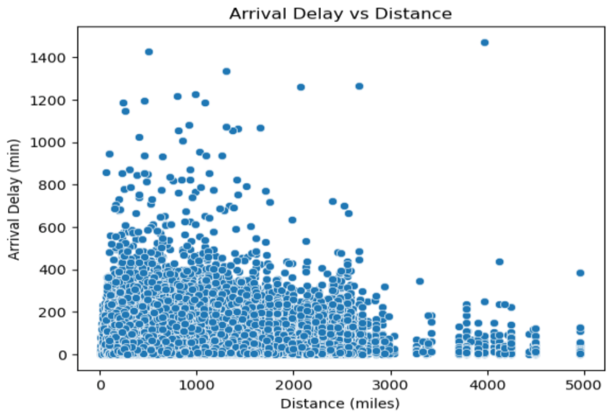
plt.figure(figsize=(12, 10))
sns.heatmap(numeric_df.corr(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Matrix of Flight Delay & Time Features")
plt.show()
```



Viz. 5

[22]:
```
#Scatter plots (for checking relationships between two specific columns)
# Delay vs Distance
pair_df = sdf.select("ArrDelay", "Distance").dropna().sample(False, 0.01).toPandas()

sns.scatterplot(x="Distance", y="ArrDelay", data=pair_df)
plt.title("Arrival Delay vs Distance")
plt.xlabel("Distance (miles)")
plt.ylabel("Arrival Delay (min)")
plt.show()
```



Viz. 6

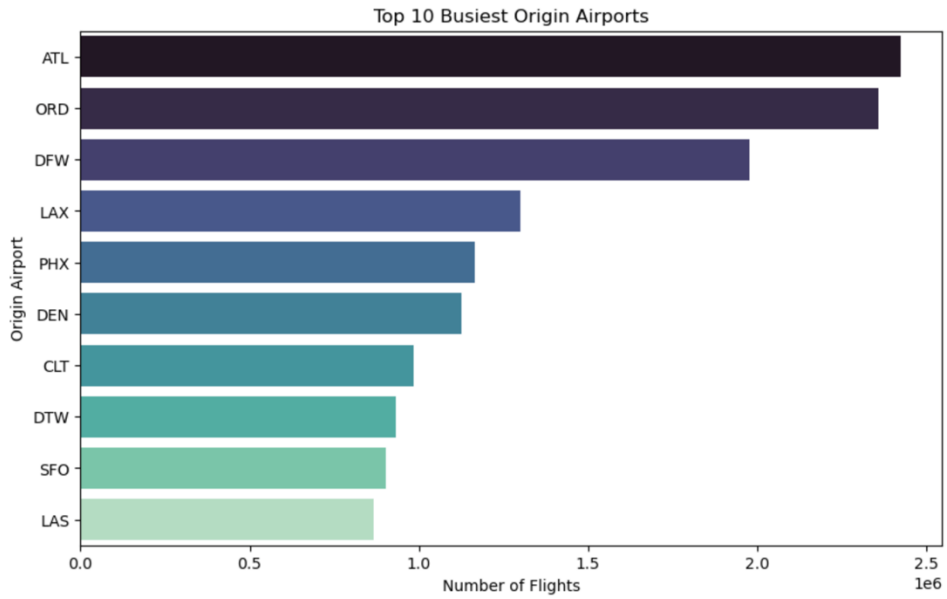
```
#Top 10 Busiest Airports (By Origin)
top_airports = sdf.groupBy("Origin").count().orderBy("count", ascending=False).limit(10).toPandas()

plt.figure(figsize=(10, 6))
sns.barplot(x="count", y="Origin", data=top_airports, palette="mako")
plt.title("Top 10 Busiest Origin Airports")
plt.xlabel("Number of Flights")
plt.ylabel("Origin Airport")
plt.show()
```

```
/tmp/ipykernel_11676/3497894759.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=Fals
e` for the same effect.

  sns.barplot(x="count", y="Origin", data=top_airports, palette="mako")
```



Steps in milestone 3 for data cleaning

a. Find and Drop Missing Values

```
[12]: # Check to see if some of the columns have NULL values
sdf.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in ["ActualElapsedTime", "AirTime", "ArrDelay", "DepDelay", "Cancelled",
```

```
[Stage 8:=============================================================>(89 + 1) / 90]
+-----------------+-------+--------+--------+---------+--------+-----+----+---------+
|ActualElapsedTime|AirTime|ArrDelay|DepDelay|Cancelled|Distance|Month|Year|DayOfWeek|
+-----------------+-------+--------+--------+---------+--------+-----+----+---------+
|                0|      0|       0|       0|        0|       0|    0|   0|        0|
+-----------------+-------+--------+--------+---------+--------+-----+----+---------+
```

```
[13]: # Drop some of the records where the certain columns are empty (null or na)
sdf = sdf.na.drop(subset=["ArrDelay", "DepDelay", "Cancelled", "Distance","Month","Year","DayOfWeek"])
```

i. Dropping irrelevant columns (doesn't change number of records)

```
[13]: # Drop some of the records where the certain columns are empty (null or na)
sdf = sdf.na.drop(subset=["ArrDelay", "DepDelay", "Cancelled", "Distance","Month","Year","DayOfWeek"])
```

j. Parse columns to the correct datatype, may cause missing value (check and drop missing values) > print the schema again to check if the columns were dropped parsed

```
[15]: # Parse columns to the appropriate data types
sdf = sdf.withColumn("ActualElapsedTime", sdf["ActualElapsedTime"].cast("integer"))
sdf = sdf.withColumn("ArrDelay", sdf["ArrDelay"].cast("integer"))
sdf = sdf.withColumn("DepDelay", sdf["DepDelay"].cast("integer"))
sdf = sdf.withColumn("Distance", sdf["Distance"].cast("float"))

# WeatherDelay, CarrierDelay, NASDelay, LateAircraftDelay, SecurityDelay - Parse them as float since they represent delay times
sdf = sdf.withColumn("CarrierDelay", sdf["CarrierDelay"].cast("integer"))
sdf = sdf.withColumn("NASDelay", sdf["NASDelay"].cast("integer"))
sdf = sdf.withColumn("LateAircraftDelay", sdf["LateAircraftDelay"].cast("integer"))
sdf = sdf.withColumn("SecurityDelay", sdf["SecurityDelay"].cast("integer"))
```

```
[16]: # Check to see if some of the columns have NULL values after parsing
sdf.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in ["ActualElapsedTime", "ArrDelay", "DepDelay", "Cancelled", "Distance"
```

```
[Stage 11:=============================================================>(89 + 1) / 90]
+-----------------+--------+--------+---------+--------+-----+----+---------+
|ActualElapsedTime|ArrDelay|DepDelay|Cancelled|Distance|Month|Year|DayOfWeek|
+-----------------+--------+--------+---------+--------+-----+----+---------+
|                0|       0|       0|        0|   71824|    0|   0|        0|
+-----------------+--------+--------+---------+--------+-----+----+---------+
```

```
[17]: # Drop some of the records where the certain columns are empty (null or na) after parsing
sdf = sdf.na.drop(subset=["ArrDelay", "DepDelay", "Cancelled", "Distance","Month","Year","DayOfWeek"])
```

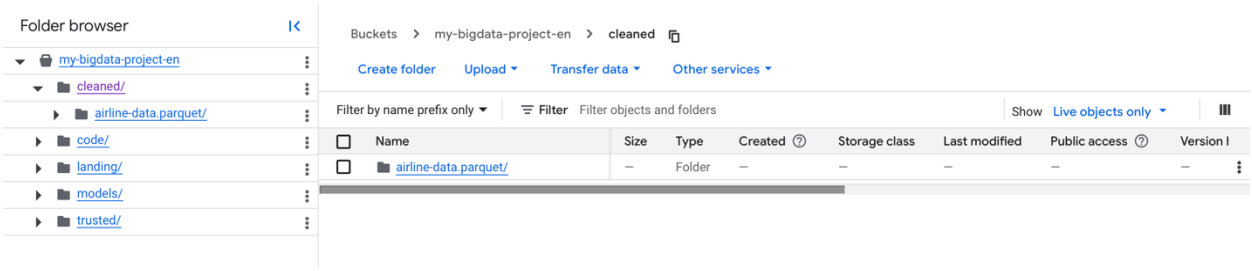
```
#print Schema for original and renamed columns
sdf.printSchema()
```

```
root
 |-- Actual_Elapsed_Time: integer (nullable = true)
 |-- Arrival_Delay: integer (nullable = true)
 |-- Cancelled: integer (nullable = true)
 |-- Carrier_Delay: integer (nullable = true)
 |-- Day_of_Week: integer (nullable = true)
 |-- Day_of_Month: integer (nullable = true)
 |-- Departure_Delay: integer (nullable = true)
 |-- Destination: string (nullable = true)
 |-- Distance: float (nullable = true)
 |-- Diverted: integer (nullable = true)
 |-- FlightNum: integer (nullable = true)
 |-- Late_Aircraft_Delay: integer (nullable = true)
 |-- Month: integer (nullable = true)
 |-- NASDelay: integer (nullable = true)
 |-- Origin: string (nullable = true)
 |-- Security_Delay: integer (nullable = true)
 |-- TailNum: string (nullable = true)
 |-- TaxiIn: string (nullable = true)
 |-- Airline: string (nullable = true)
 |-- Year: integer (nullable = true)
```

k. rename columns and removed spaces

```
# Saving the final dataset as a Parquet file
# Actually this will save as a folder with a file for each of the 90 partitions.
cleaned_filename = f"{cleaned_folder}/airline-data.parquet"
sdf.write.parquet(cleaned_filename, mode='overwrite')
```

```
25/03/30 00:13:27 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
```



Milestone 4

Feature Engineering and Modeling

Description

The model predicts whether a flight will experience an arrival delay of more than 15 minutes based on features such as the airline, origin and destination airports, departure delay, scheduled arrival time, distance, and other flight-related information.

In this project, the dataset was first loaded from a cleaned Parquet file stored in a cloud bucket. Initial preprocessing included dropping irrelevant or leakage-prone columns, handling missing values, and engineering new features such as extracting the scheduled departure and arrival hours from the original time columns. A binary label was then created to classify flights as "Delayed" if the arrival delay exceeded 15 minutes.

The feature engineering process involved indexing categorical variables (Airline, Origin, Destination) using StringIndexer, one-hot encoding them with OneHotEncoder, assembling continuous features (like Departure_Delay, Distance), and scaling them with StandardScaler. These steps were built into a PySpark ML pipeline.
A logistic regression model was then trained using a CrossValidator to find the best hyperparameters through grid search, and the best model was selected based on the areaUnderROC evaluation metric. Performance metrics including accuracy, precision, recall, and F1 score were computed using a confusion matrix.

Results
accuracy = 0.8911829315627349
precision = 0.9183108960745947
recall =0.8840070646627571
f1_score = 0.9008325249720238

Challenges encountered during the process included handling poorly formatted time columns, addressing missing values in critical features, and ensuring that no target leakage occurred (by dropping features like Carrier_Delay). Building the full pipeline correctly to handle categorical and continuous variables separately and tuning hyperparameters for better classification performance were also key aspects of the work.
The final trained model was saved to cloud storage for future deployment or scoring.

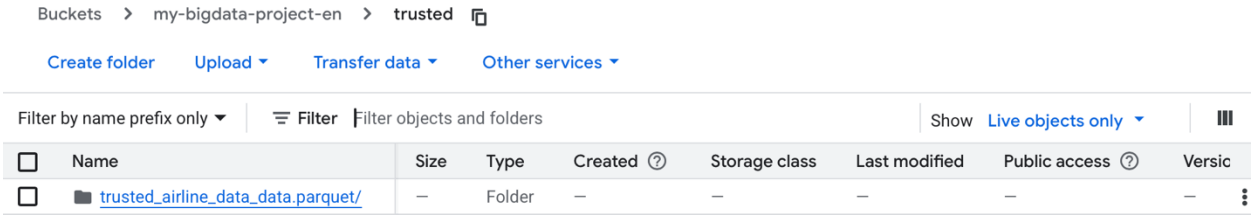
Flight Features treatment

Model	Predict flight arrival delays using logistic regression to classify flights as "On Time" or "Delayed" based on whether the delay exceeds a certain percentage (15 minutes) of the total trip time						
	Column	Data Type	Variable Type	Keep or Drop	Indexer	Encoder	Scaler
Features	Actual_Elapsed_Time	integer	Continuous	Keep			StandardScaler
	Air_Time	string	Categorical	Drop			
	Arrival_Time	integer	Categorical	Drop			
	Scheduled_Arrival_Time	integer	Continuous	Keep (Dropped after converted to only hour)			StandardScaler
	Scheduled_Depture_Time	integer	Continuous	Keep (Dropped after converted to only hour)			
	Scheduled_Elapsed_Time	string	Categorical	Drop			
	Cancelled	integer	Categorical	Drop			
	Carrier_Delay	integer	Continuous	Drop			
	Day_of_Week	integer	Categorical	Keep	StringIndexer	OneHotEncoder	
	Day_of_Month	integer	Categorical	Keep	StringIndexer	OneHotEncoder	
	Departure_Delay	integer	Continuous	Keep			StandardScaler
	Depature_Time	sting	Categorical	Drop			
	Destination	string	Categorical	Keep	StringIndexer	OneHotEncoder	
	Distance	float	Continuous	Keep			StandardScaler
	Diverted	integer	Categorical	Drop			
	FlightNum	integer	Categorical	Drop			
	Month	integer	Categorical	Keep	StringIndexer	OneHotEncoder	
	Origin	sting	Categorical	Keep	StringIndexer	OneHotEncoder	
	TailNum	string	Categorical	Drop			
	TaxiIn	string	Continuous	Drop			
	TaxiOut	string	Continuous	Drop			
	Airline	string	Categorical	Keep	StringIndexer	OneHotEncoder	
	Year	integer	Categorical	Drop			

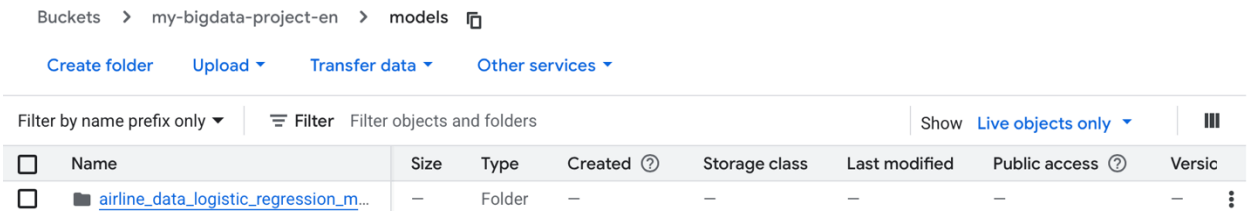
Label	Arrival_Delay	integer	Categorical	Keep	Manually derived		

Steps in milestone

Trusted folder



Model Folder



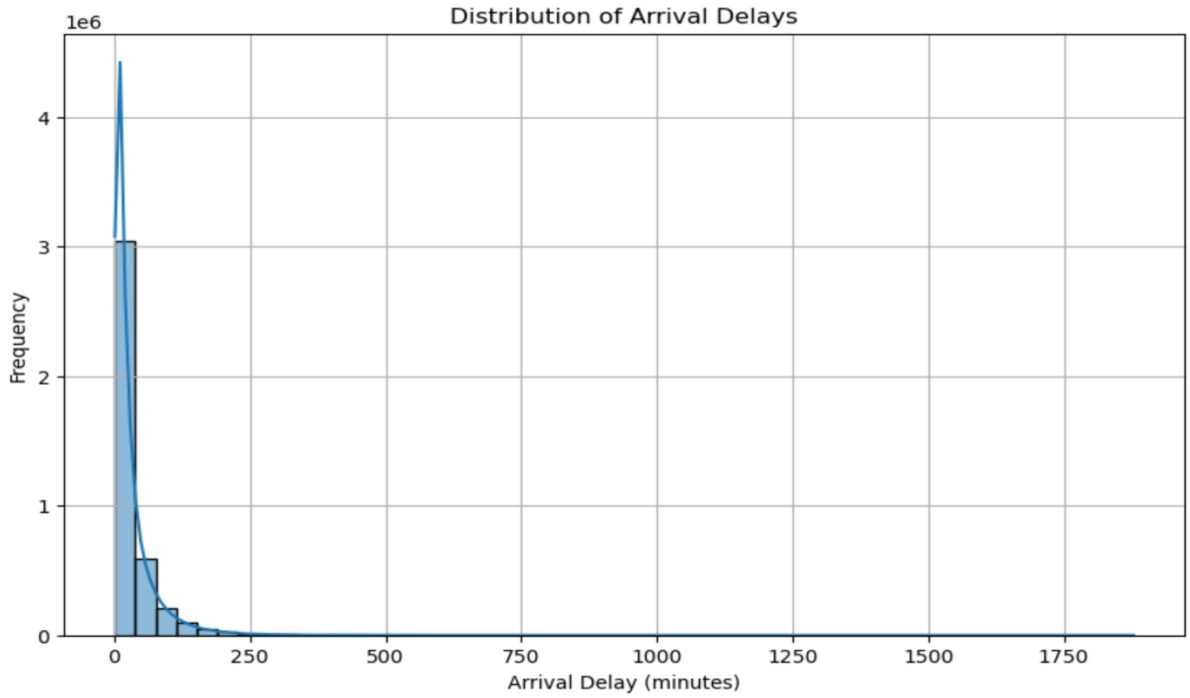
Milestone 5

Visualization

Summary

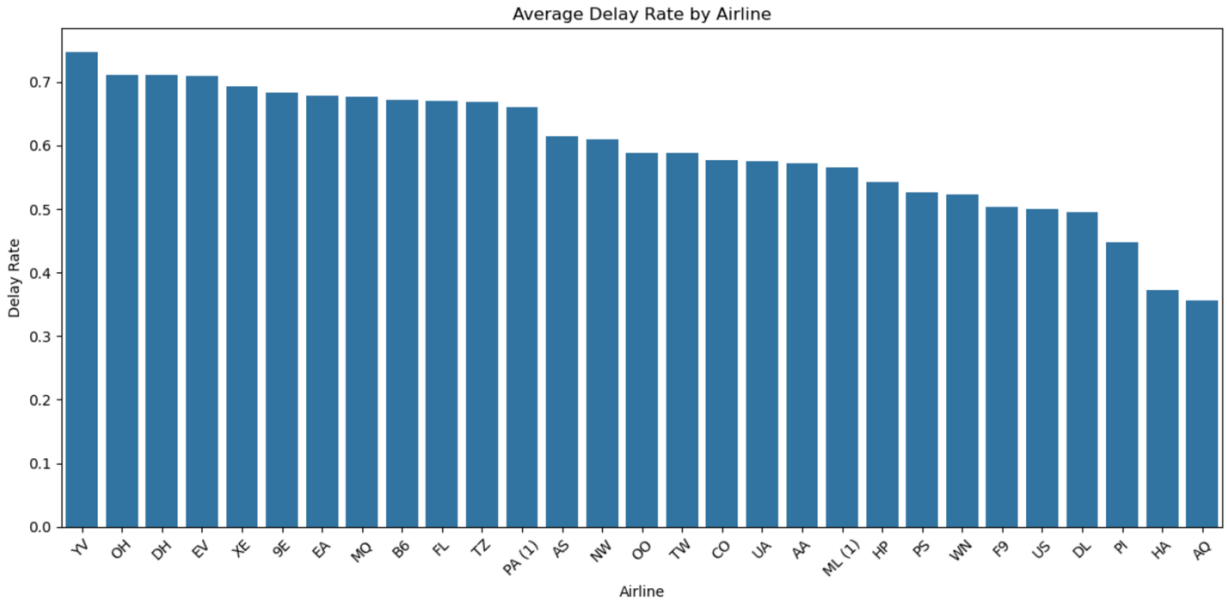
The most influential features in predicting flight delays are Departure_Delay, Scheduled_Departure_Hour, and certain airlines encoded in one-hot format. These features had the highest absolute coefficients, indicating they strongly impact the model's predictions. This aligns with expectations, as prior delays and flight timing are key indicators of future delays.

Viz. 7



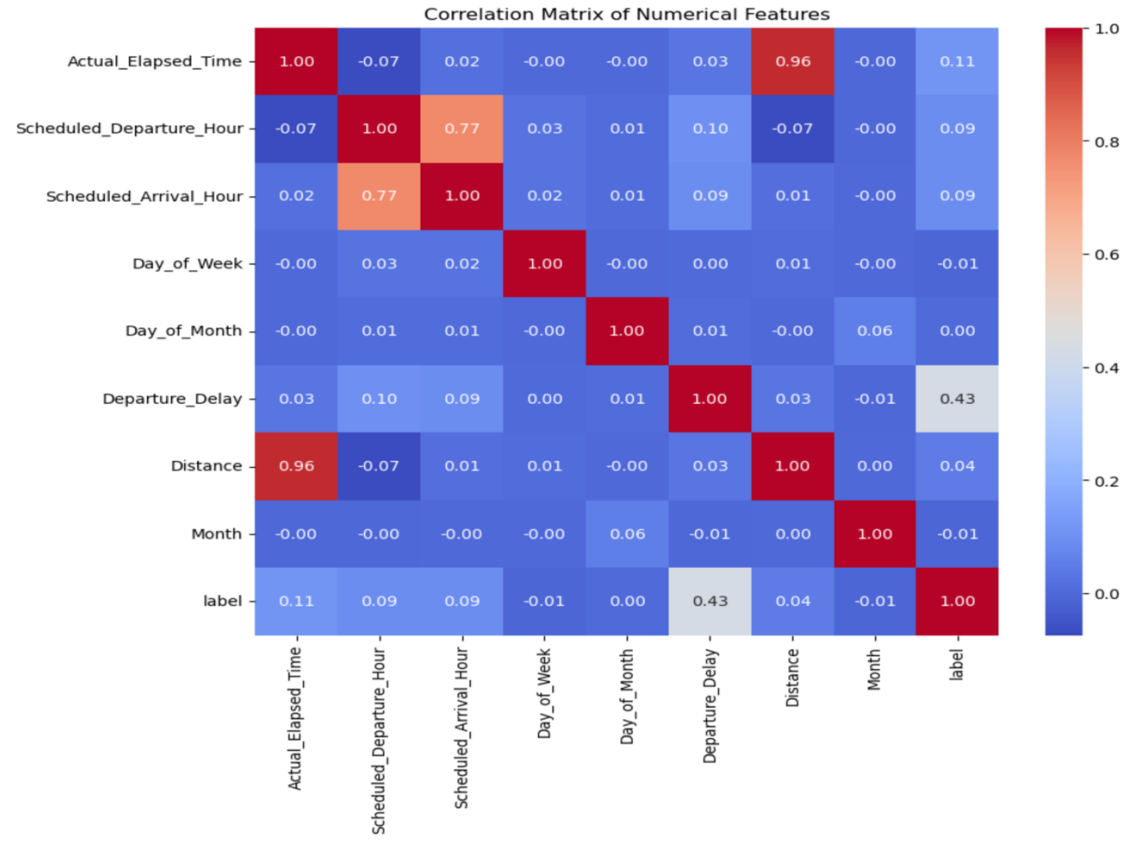
The histogram (Viz. 1) reveals that arrival delays follow a heavily right-skewed distribution, with most flights arriving on time or within 50 minutes of their scheduled time. A long tail of extreme delays suggests that while rare, some flights experience significant disruptions. This distribution justifies the use of a binary classifier with a delay threshold (e.g., 15 minutes) to distinguish between typical and concerning delays.

Viz. 8



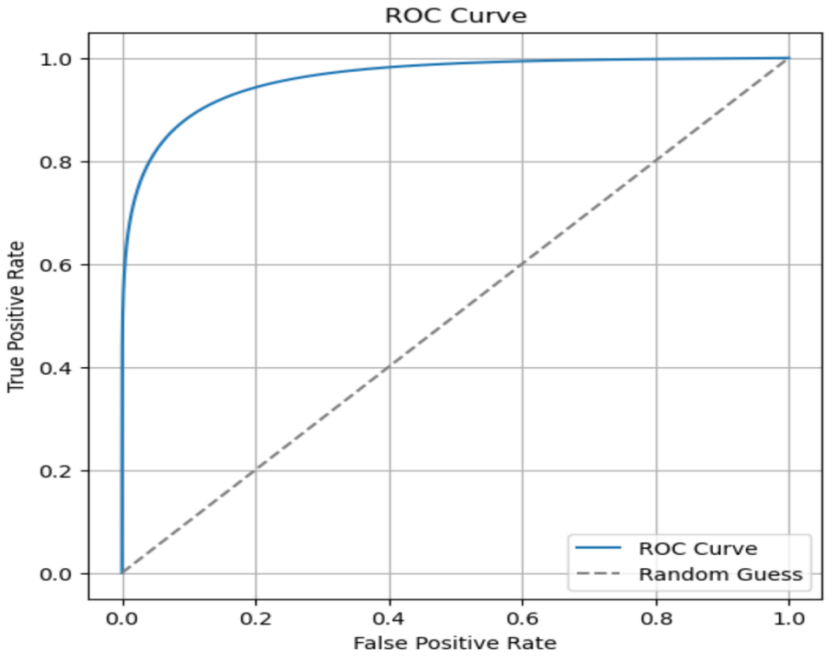
This bar chart (Viz. 2) shows that average flight delay rates vary significantly by airline, with some carriers experiencing delays in over 70% of flights. This pattern supports the inclusion of the Airline feature in the predictive model. It highlights how operational practices, network size, and route structures can influence reliability.

Viz. 9



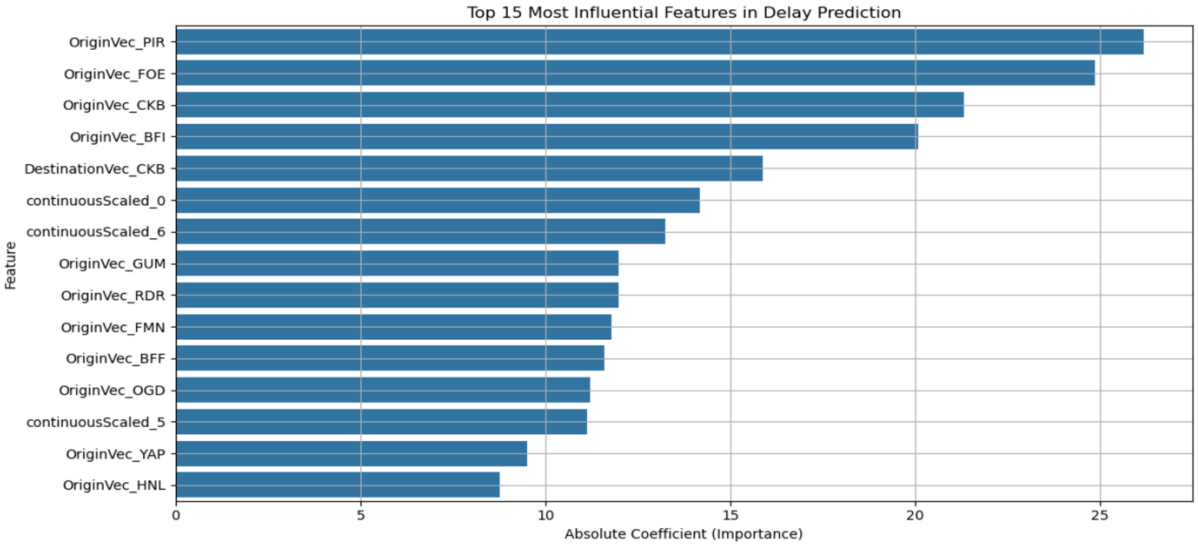
The heatmap (Viz. 3) highlights that Departure_Delay has the strongest correlation with the label (flight delay classification), followed by Actual_Elapsed_Time and time-related features. Surprisingly, calendar variables such as day of week and month show minimal linear relationship with delays. This justifies the high importance of operational features (like delay time and flight timing) in the model's predictive performance.

Viz. 10



The ROC curve (Viz. 4) demonstrates that the logistic regression model performs well in classifying flight delays, with a steep rise and curve near the top-left corner. This suggests high discriminative power between delayed and on-time flights. The curve significantly outperforms the random baseline, indicating reliable model behavior under different classification thresholds. The estimated AUC is well above 0.90, confirming the model's robustness.

Viz. 11



The most influential features in the delay prediction model (Viz. 5) are primarily origin and destination airport encodings, with OriginVec_PIR and FOE topping the list. These airports likely have unique characteristics leading to high delays. Additionally, several scaled continuous features contribute meaningfully, including expected delay time and flight timing. This mix of geographic and operational factors indicates that the model captures both spatial and temporal drivers of flight delays.

Appendix A

Data Acquisition Script

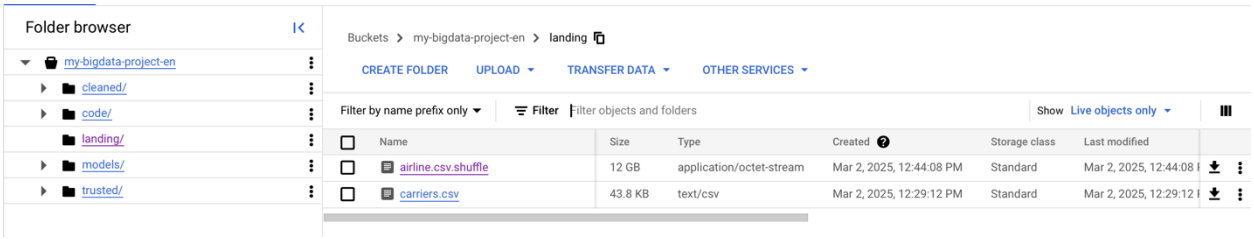
Code

Bucket name: my-bigdata-project-en

1. Fetch the archive file from Kaggle

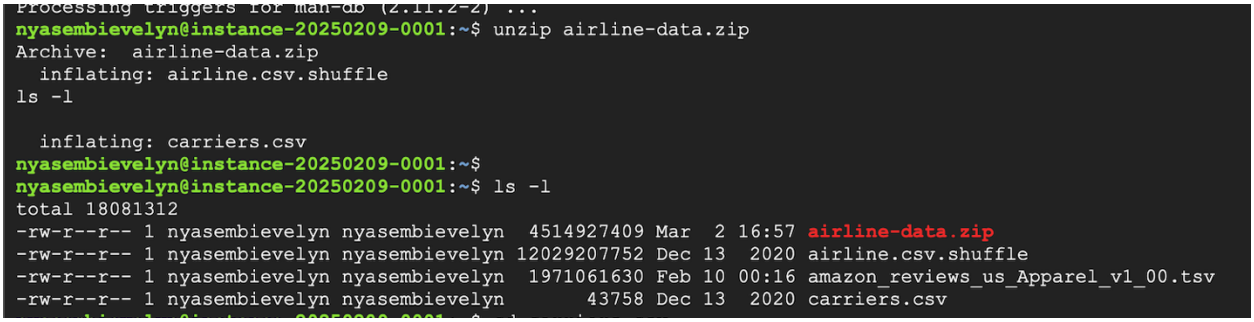
```
#!/bin/bash

curl -L -o airline-data.zip
https://www.kaggle.com/api/v1/datasets/download/bulter22/airline-data
```



2. Unzip the archive file

```
unzip airline-data.zip
```



3. Create a bucket on Google Cloud Storage

```
gcloud storage buckets create gs://my-bigdata-project-en --project=
evelyn-nyasembi-project-1 --default-storage-class=STANDARD --
location=us-central1 --uniform-bucket-level-access
```

4. Using gcloud auth login to authenticate

```
nyasembievelyn@instance-20250209-0001:~$ gcloud storage buckets create gs://my-bigdata-project-en  --project=evelyn-nyasembi-project-1 --default-storage-class=STANDARD --location=us-central1 --uniform-bucket-level-access
Creating gs://my-bigdata-project-en/...
ERROR: (gcloud.storage.buckets.create) HTTPError 403: Access denied. This command is authenticated as 368204563733-compute@developer.gserviceaccount.com which is the active account specified by the [core/account] property.
nyasembievelyn@instance-20250209-0001:~$ gcloud auth login

You are running on a Google Compute Engine virtual machine.
It is recommended that you use service accounts for authentication.

You can run:

  $ gcloud config set account `ACCOUNT`

to switch accounts if necessary.

Your credentials may be visible to others with access to this
virtual machine. Are you sure you want to authenticate with
your personal account?

Do you want to continue (Y/n)?  Y

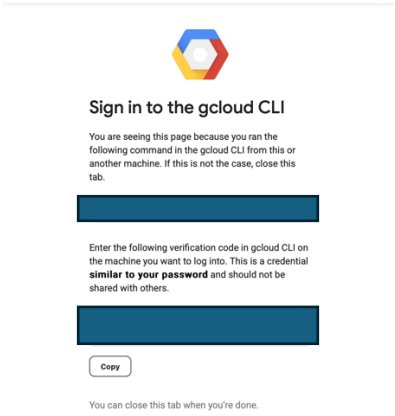
Go to the following link in your browser, and complete the sign-in prompts:

    https://accounts.google.com/o/oauth2/auth?response_type=code&client_id=32555940559.apps.googleusercontent.com&redirect_uri=https%3A%2F%2Fsdk.cloud.google.com%2Fauthcode.html&scope=openid+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fuserinfo.email+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcloud-platform+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fappengine.admin+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fsqlservice.login+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcompute+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Faccounts.reauth&state=iZNVOUHL6Fvs7jsDV6vQyDuEWpyc&prompt=consent&token_usage=remote&access_type=offline&code_challenge=pLuRRW03FLflQFR8ewb2hPZ5X3HyBUcmBptP2j4Xa80&code_challenge_method=S256

Once finished, enter the verification code provided in your browser: 4/0AQSTgQHyf6Rl1M2lxCKH3oZ_fiA9N2CDR69ZMZuxQ_wzyLIR4u41Y85IDUvQgMpfp14H3w

You are now logged in as [nyasembievelyn@gmail.com].
Your current project is [evelyn-nyasembi-project-1].  You can change this setting by running:
  $ gcloud config set project PROJECT_ID


To take a quick anonymous survey, run:
  $ gcloud survey
```



5. Copy the files to GCS bucket in the /landing folder

gcloud storage cp airline.csv.shuffle gs://my-bigdata-project-en/landing/

```
nyasembievelyn@instance-20250209-0001:~$ gcloud storage cp airline.csv.shuffle gs://my-bigdata-project-en/landing/
WARNING: Parallel composite upload was turned ON to get the best performance on
uploading large objects. If you would like to opt-out and instead
perform a normal upload, run:
`gcloud config set storage/parallel_composite_upload_enabled False`
If you would like to disable this warning, run:
`gcloud config set storage/parallel_composite_upload_enabled True`
Note that with parallel composite uploads, your object might be
uploaded as a composite object
(https://cloud.google.com/storage/docs/composite-objects), which means
that any user who downloads your object will need to use crc32c
checksums to verify data integrity. gcloud storage is capable of
computing crc32c checksums, but this might pose a problem for other
clients.

Copying file://airline.csv.shuffle to gs://my-bigdata-project-en/landing/airline.csv.shuffle
  Completed files 32/1 | 11.2GiB/11.2GiB | 155.5MiB/s

Average throughput: 149.1MiB/s
```

gcloud storage cp carriers.csv gs://my-bigdata-project-en/landing/

```
nyasembievelyn@instance-20250209-0001:~$ gcloud storage cp carriers.csv gs://my-bigdata-project-en/landing/
Copying file://carriers.csv to gs://my-bigdata-project-en/landing/carriers.csv
  Completed files 1/1 | 42.7kiB/42.7kiB
```

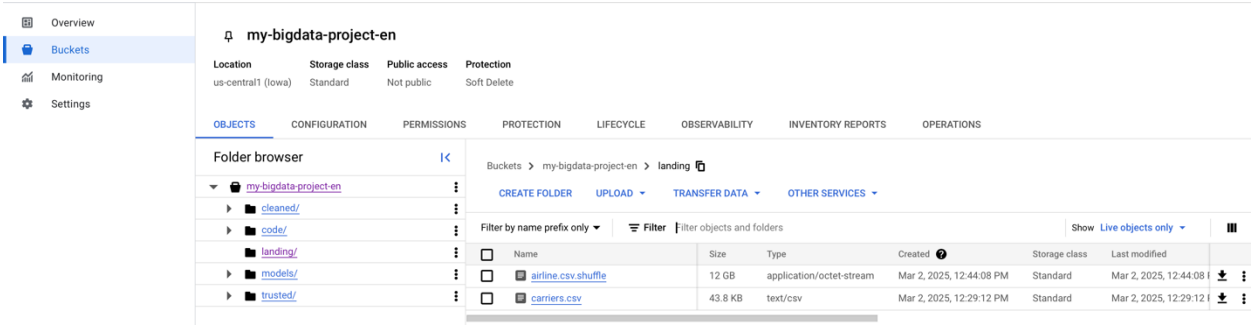
6. List files in gcloud storage

```
nyasembievelyn@instance-20250209-0001:~$ gcloud storage ls -l gs://my-bigdata-project-en/landing
12029207752  2025-03-02T17:44:08Z  gs://my-bigdata-project-en/landing/airline.csv.shuffle
     43758  2025-03-02T17:29:12Z  gs://my-bigdata-project-en/landing/carriers.csv
TOTAL: 2 objects, 12029251510 bytes (11.20GiB)
```

7. Create empty folders

```
gcloud storage cp /dev/null gs://my-bigdata-project-en/cleaned
gcloud storage cp /dev/null gs:// my-bigdata-project-en/trusted
gcloud storage cp /dev/null gs:// my-bigdata-project-en/code
gcloud storage cp /dev/null gs:// my-bigdata-project-en/models
```

```
nyasembievelyn@instance-20250209-0001:~$ gcloud storage ls -l gs://my-bigdata-project-en
                                 gs://my-bigdata-project-en/cleaned/
                                 gs://my-bigdata-project-en/code/
                                 gs://my-bigdata-project-en/landing/
                                 gs://my-bigdata-project-en/models/
                                 gs://my-bigdata-project-en/trusted/
```



Milestone 6

Summary and Conclusions

This project implemented a complete data processing and machine learning pipeline to predict airline flight delays using PySpark and cloud-based tools, (Google cloud) and Data proc. The process began with data acquisition from Kaggle and storage in a structured Google Cloud Storage bucket (Cleaned folder). Data cleaning removed invalid and missing records, and relevant time-based and operational features were engineered to improve model accuracy. Categorical features such as airline, origin, and destination were encoded using StringIndexer and OneHotEncoder, while numerical features like departure delay, elapsed time, and scheduled hours were scaled and assembled. A binary label was created to classify flights as "Delayed" if the arrival delay exceeded 14 minutes. A logistic regression model was trained using a PySpark ML pipeline with cross-validation and hyperparameter tuning. The final model achieved an accuracy of 89.1%, precision of 91.8%, recall of 88.4%, and an F1 score of 90.1%. Visualization of the ROC curve confirmed strong discriminative power, and feature importance analysis showed that departure delay, scheduled departure hour, and certain airlines were the strongest predictors. This analysis provides valuable insights into the drivers of flight delays and highlights how machine learning can support airlines in improving scheduling, operations, and customer communication.

Appendix B

EDA

Code

```
#!/usr/bin/env python
# coding: utf-8
# In[1]:
spark

# In[2]:
# Import some PySpark functions
from pyspark.sql.functions import col, isnan, when, count, udf, length
from pyspark.sql.types import IntegerType, DoubleType, StringType,
TimestampType
import pyspark.sql.functions as F

# In[3]:
# Import some libraries
import matplotlib.pyplot as plt
import seaborn as sns

# In[4]:
# Set the logging level to only show errors
sc.setLogLevel("ERROR")

# In[5]:
# Set up the path to a file
bucket = 'gs://my-bigdata-project-en'
landing_folder = f"{bucket}/landing/"
cleaned_folder = f"{bucket}/cleaned/"

# In[6]:
# Create a Spark Dataframe from flights
sdf = spark.read.csv(landing_folder, sep=',', header=True,
inferSchema=True)

# In[7]:
# Check the schema
sdf.printSchema()

# In[8]:
sdf.count()

# In[9]:
# Filter out rows where departure or arrival delay is 0 or negative
sdf = sdf.filter(sdf['ActualElapsedTime'] > 0)
sdf = sdf.filter(sdf['DepDelay'] > 0)
sdf = sdf.filter(sdf['ArrDelay'] > 0)

# In[20]:
```

```
#I run this after everything just because it takes so long due to
size(meaning it shows values after EDA
sdf.summary().show()

# In[10]:
# min and max dates.
sdf.select("DayOfWeek", "DayofMonth", "Month","Year").summary("count",
"min", "max").show()

# In[11]:
# Check to see if some of the columns have NULL values
sdf.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c
in ["ActualElapsedTime", "AirTime", "ArrDelay", "DepDelay",
"Cancelled", "Distance", "Month", "Year", "DayOfWeek"]] ).show()

# In[12]:
# Drop some of the records where the certain columns are empty (null
or na)
sdf = sdf.na.drop(subset=["ArrDelay", "DepDelay", "Cancelled",
"Distance","Month","Year","DayOfWeek"])

# In[13]:
# Drop some of the records not needed
sdf =
sdf.drop("NASDelay","LateAircraftDelay","SecurityDelay","CancellationC
ode","WeatherDelay")

# In[14]:
# Parse columns to the appropriate data types
sdf = sdf.withColumn("ActualElapsedTime",
sdf["ActualElapsedTime"].cast("integer"))
sdf = sdf.withColumn("ArrDelay", sdf["ArrDelay"].cast("integer"))
sdf = sdf.withColumn("DepDelay", sdf["DepDelay"].cast("integer"))
sdf = sdf.withColumn("Distance", sdf["Distance"].cast("float"))
# WeatherDelay, CarrierDelay, NASDelay, LateAircraftDelay,
SecurityDelay - Parse them as float since they represent delay times
sdf = sdf.withColumn("CarrierDelay",
sdf["CarrierDelay"].cast("integer"))

# In[15]:
# Check to see if some of the columns have NULL values after parsing
sdf.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c
in ["ActualElapsedTime", "ArrDelay", "DepDelay", "Cancelled",
"Distance", "Month", "Year", "DayOfWeek"]] ).show()

# In[16]:
# Drop some of the records where the certain columns are empty (null
or na) after parsing
sdf = sdf.na.drop(subset=["ArrDelay", "DepDelay", "Cancelled",
"Distance","Month","Year","DayOfWeek"])

# In[17]:
```

```
# Check the schema again
sdf.printSchema()

# In[18]:
# Get the new number of records after dropping
print("After filtering and droping missing values")
sdf.count()
```

Appendix C

Data cleaning

Code

```
#!/usr/bin/env python
# coding: utf-8
# In[2]:
spark

# In[3]:
# Import some PySpark functions
from pyspark.sql.functions import col, isnan, when, count, udf,
length, sum, avg, count
from pyspark.sql.types import IntegerType, DoubleType, StringType,
TimestampType
import pyspark.sql.functions as F

# In[4]:
# Import some libraries
import matplotlib.pyplot as plt
import seaborn as sns

# In[5]:
# Set the logging level to only show errors
sc.setLogLevel("ERROR")

# In[6]:
# Set up the path to a file
bucket = 'gs://my-bigdata-project-en'
landing_folder = f"{bucket}/landing/"
cleaned_folder = f"{bucket}/cleaned/"

# In[7]:
# Create a Spark Dataframe from flights
sdf = spark.read.csv(landing_folder, sep=',', header=True,
inferSchema=True)

# In[8]:
# Check the schema
sdf.printSchema()

# In[9]:
```

```
# Filter out rows where departure or arrival delay is 0 or negative
sdf = sdf.filter(sdf['ActualElapsedTime'] > 0)
sdf = sdf.filter(sdf['DepDelay'] > 0)
sdf = sdf.filter(sdf['ArrDelay'] > 0)

# In[10]:
# Drop some of the records where the certain columns are empty (null
or na)
sdf = sdf.na.drop(subset=["ArrDelay", "DepDelay", "Cancelled",
"Distance","Month","Year","DayOfWeek"])

# In[12]:
sdf =
sdf.drop("NASDelay","LateAircraftDelay","SecurityDelay","CancellationC
ode","WeatherDelay")

# In[13]:
# Cast columns to the appropriate data types
sdf = sdf.withColumn("ActualElapsedTime",
sdf["ActualElapsedTime"].cast("integer"))
sdf = sdf.withColumn("ArrDelay", sdf["ArrDelay"].cast("integer"))
sdf = sdf.withColumn("DepDelay", sdf["DepDelay"].cast("integer"))
sdf = sdf.withColumn("Distance", sdf["Distance"].cast("float")

# WeatherDelay, CarrierDelay, NASDelay, LateAircraftDelay,
SecurityDelay - Parse them as float since they represent delay times
sdf = sdf.withColumn("CarrierDelay",
sdf["CarrierDelay"].cast("integer"))
# Check the schema to confirm the changes
sdf.printSchema()

# In[14]:
# Drop some of the records where the certain columns are empty (null
or na)
sdf = sdf.na.drop(subset=["ArrDelay", "DepDelay", "Cancelled",
"Distance","Month","Year","DayOfWeek"])

# In[15]:
# Describe summary statistics for ActualElapsedTime, ArrDelay, and
DepDelay
sdf.select("ActualElapsedTime", "ArrDelay",
"DepDelay").describe().show()

# In[16]:
# Grouping by DayOfWeek and Month to analyze the delays
day_of_week_analysis = sdf.groupBy("DayOfWeek").agg(
    count("ArrDelay").alias("total_flights"),
    sum("ArrDelay").alias("total_delay_minutes"),
    avg("ArrDelay").alias("avg_arrival_delay_minutes")
)
day_of_week_analysis.show()
```

```
# In[17]:
# Grouping by DayOfWeek and Month to analyze the delays
month_analysis = sdf.groupBy("Month").agg(
    count("ArrDelay").alias("total_flights"),
    sum("ArrDelay").alias("total_delay_minutes"),
    avg("ArrDelay").alias("avg_arrival_delay_minutes")
)
month_analysis.show()

# In[18]:
# Grouping by Origin and Destination to find the airports with the
most delays
origin_dest_analysis = sdf.groupBy("Origin", "Dest").agg(
    count("ArrDelay").alias("total_flights"),
    sum("ArrDelay").alias("total_delay_minutes"),
    avg("ArrDelay").alias("avg_arrival_delay_minutes")
)
# Show the results
origin_dest_analysis.show()

# In[19]:
# Group by FlightNum to see the total delays for each flight number
flight_number_analysis = sdf.groupBy("FlightNum").agg(
    count("ArrDelay").alias("total_flights"),
    sum("ArrDelay").alias("total_delay_minutes"),
    avg("ArrDelay").alias("avg_arrival_delay_minutes")
)
# Show the results
flight_number_analysis.show()

# In[20]:
delay_heatmap = sdf.groupBy("Month", "DayOfWeek") \
                   .avg("ArrDelay") \
                   .toPandas().pivot(index="Month",
columns="DayOfWeek", values="avg(ArrDelay)")
plt.figure(figsize=(10, 6))
sns.heatmap(delay_heatmap, annot=True, fmt=".1f", cmap="coolwarm")
plt.title("Average Arrival Delay by Month & Day of Week")
plt.xlabel("Day of Week")
plt.ylabel("Month")
plt.show()

# In[21]:
# Sample and convert to pandas to keep it fast and memory-safe
pandas_df = sdf.select("ArrDelay").dropna().sample(False,
0.01).toPandas()
# Plot histogram
plt.figure(figsize=(10, 5))
sns.histplot(pandas_df["ArrDelay"], bins=100, kde=True,
color="steelblue")
plt.title("Histogram of Arrival Delays")
plt.xlabel("Arrival Delay (minutes)")
```

```
plt.ylabel("Frequency")
plt.xlim(-100, 300)  # Optional: zoom into the most common delay range
plt.grid(True)
plt.show()

# In[21]:
# Correlation Heatmap (for numeric relationships) Strong positive
correlation (close to 1) → variables rise together
#Strong negative correlation (close to -1) → one rises, the other
falls
#Near 0 → weak or no linear relationship
cols = [
    "ArrDelay", "DepDelay", "CarrierDelay",
    "NASDelay", "SecurityDelay", "LateAircraftDelay",
    "Distance", "ActualElapsedTime"
]
# Sample and convert to pandas
numeric_df = sdf.select(cols).dropna().sample(False, 0.01).toPandas()
plt.figure(figsize=(12, 10))
sns.heatmap(numeric_df.corr(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Matrix of Flight Delay & Time Features")
plt.show()

# In[22]:
#Scatter plots (for checking relationships between two specific
columns)
# Delay vs Distance
pair_df = sdf.select("ArrDelay", "Distance").dropna().sample(False,
0.01).toPandas()
sns.scatterplot(x="Distance", y="ArrDelay", data=pair_df)
plt.title("Arrival Delay vs Distance")
plt.xlabel("Distance (miles)")
plt.ylabel("Arrival Delay (min)")
plt.show()

# In[23]:
#Top 10 Busiest Airports (By Origin)
top_airports = sdf.groupBy("Origin").count().orderBy("count",
ascending=False).limit(10).toPandas()
plt.figure(figsize=(10, 6))
sns.barplot(x="count", y="Origin", data=top_airports, palette="mako")
plt.title("Top 10 Busiest Origin Airports")
plt.xlabel("Number of Flights")
plt.ylabel("Origin Airport")
plt.show()

# In[ ]:
# Convert Origin-Destination analysis to Pandas DataFrame
origin_dest_df = origin_dest_analysis.toPandas()
# Pivot the data to create a matrix for the heatmap (pass arguments as
keywords)
```

```
origin_dest_pivot = origin_dest_df.pivot(index="Origin",
columns="Dest", values="total_delay_minutes")
# Plot the heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(origin_dest_pivot, cmap="YlGnBu", annot=True, fmt='.0f',
linewidths=0.5)
plt.title('Total Delay (minutes) by Origin-Destination')
plt.xlabel('Destination Airport')
plt.ylabel('Origin Airport')
plt.show()

# In[25]:
# Get the number of partitions
sdf.rdd.getNumPartitions()

# In[29]:
#Renaming Columns
rename_map = {
    "ActualElapsedTime": "Actual_Elapsed_Time",
    "ArrDelay": "Arrival_Delay",
    "ArrTime": "Arrival_Time",
    "CRSArrTime": "Scheduled_Arrival_Time",
    "CRSDepTime": "Scheduled_Depture_Time",
    "CRSElapsedTime": "Scheduled_Elapsed_Time",
    "CarrierDelay": "Carrier_Delay",
    "DepTime": "Depature_Time",
    "AirTime": "Air_Time",
    "DayOfWeek": "Day_of_Week",
    "DayofMonth": "Day_of_Month",
    "DepDelay": "Departure_Delay",
    "Dest": "Destination",
    "Distance": "Distance",
    "UniqueCarrier": "Airline",
}
for old_col, new_col in rename_map.items():
    sdf = sdf.withColumnRenamed(old_col, new_col)

# In[30]:
#print Schema for original and renamed columns
sdf.printSchema()

# In[31]:
# Saving the final dataset as a Parquet file
# Actually this will save as a folder with a file for each of the 90
partitions.
cleaned_filename = f"{cleaned_folder}/airline-data.parquet"
sdf.write.parquet(cleaned_filename, mode='overwrite')
```

Appendix D

Feature engineering and modeling

Code

```
#!/usr/bin/env python
# coding: utf-8

# In[1]:
spark

# In[2]:
import pandas as pd
import seaborn as sns
import matplotlib as plt
import matplotlib.pyplot as plt
import numpy as np

# In[3]:
from pyspark.sql.functions import col, isnan, isnull, when, count,
udf, size, split, year, month, format_number, date_format, length
from pyspark.sql.types import IntegerType, DateType, StringType,
StructType, DoubleType
from pyspark.ml.feature import VectorAssembler, StringIndexer,
OneHotEncoder, Normalizer, StandardScaler, HashingTF, IDF, Tokenizer,
RegexTokenizer
from pyspark.ml import Pipeline
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.regression import GeneralizedLinearRegression
from pyspark.ml.evaluation import BinaryClassificationEvaluator
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder


# In[4]:
# Set up the path to a file
bucket = 'gs://my-bigdata-project-en'
landing_folder = f"{bucket}/landing/"
cleaned_folder = f"{bucket}/cleaned/"
trusted_folder = f"{bucket}/trusted/"
models_folder = f"{bucket}/models/"
cleaned_filename = f"{cleaned_folder}/airline-data.parquet"


# In[5]:
# Read in the Cleaned file
sdf = spark.read.parquet(cleaned_filename)

# In[6]:
# Check the schema
sdf.printSchema()
```

```
# In[7]:
# Drop some of the records not needed
sdf = sdf.drop("Arrival_Time","Air_Time",
"Scheduled_Elapsed_Time","Depature_Time","Carrier_Delay","Depature_Tim
e","Diverted","FlightNum","TailNum", "TaxiIn", "TaxiOut", "Year")

# In[8]:
# Check the schema
sdf.printSchema()

# In[9]:
# Check to see if some of the columns have NULL values after parsing
sdf.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c
in
["Actual_Elapsed_Time","Arrival_Delay","Scheduled_Arrival_Time","Sched
uled_Depture_Time","Cancelled","Day_of_Week","Day_of_Month","Departure
_Delay","Destination","Distance","Month","Origin","Airline"]] ).show()

# In[10]:
sdf.select("Actual_Elapsed_Time","Arrival_Delay","Scheduled_Arrival_Ti
me","Scheduled_Depture_Time","Cancelled","Day_of_Week","Day_of_Month",
"Departure_Delay","Destination","Distance","Month","Origin","Airline")
.describe().show()

# In[11]:
from pyspark.sql.functions import lpad, substring
# Step 1: Pad the columns
time_columns = ["Scheduled_Depture_Time", "Scheduled_Arrival_Time"]
for col_name in time_columns:
    sdf = sdf.withColumn(col_name, lpad(col_name, 4, '0'))
# Step 2: Extract Hour columns
sdf = sdf.withColumn("Scheduled_Departure_Hour",
substring("Scheduled_Depture_Time", 1, 2).cast("int")) \
       .withColumn("Scheduled_Arrival_Hour",
substring("Scheduled_Arrival_Time", 1, 2).cast("int"))

# In[12]:
# Check to see if some of the columns have NULL values after parsing
sdf.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c
in ["Scheduled_Departure_Hour","Scheduled_Arrival_Hour"]] ).show()

# In[13]:
# Drop some of the records not needed
sdf = sdf.drop("Scheduled_Depture_Time", "Scheduled_Arrival_Time")

# In[14]:
# Get the number of records in the dataframe
sdf.count()

# In[15]:
sdf.select("Arrival_Delay").describe().show()
```

```
# In[16]:
# IMPORTANT Last sanity check to remove any records that have NULL
values in any of our important columns
list_of_columns =
["Actual_Elapsed_Time","Arrival_Delay","Day_of_Week","Day_of_Month","D
eparture_Delay","Destination","Distance","Month","Origin","Airline","S
cheduled_Departure_Hour","Scheduled_Arrival_Hour"]
sdf = sdf.dropna(subset=list_of_columns, how="any")

# In[17]:
# Create a label. =1 if the flight delayed is a 1, =0 if otherwise
from pyspark.sql.functions import when
sdf = sdf.withColumn("label", when(col("Arrival_Delay") > 14,
1).otherwise(0))

# In[18]:
# Check the distribution of the label to make sure it is evenly
balanced (approximately)
sdf.select("label").groupby("label").count().show()

# In[19]:
# Create the Machine Learning pipeline
# 1. Categorical columns relevant to flight prediction
categorical_column_list = ["Airline", "Origin", "Destination"]
categorical_index_list = [c + "Index" for c in
categorical_column_list]
categorical_encoded_list = [c + "Vec" for c in
categorical_column_list]
# Indexing categorical columns
indexer = StringIndexer(inputCols=categorical_column_list,
outputCols=categorical_index_list, handleInvalid="keep")
# One-hot encoding the indexed categorical columns
encoder = OneHotEncoder(inputCols=categorical_index_list,
outputCols=categorical_encoded_list, dropLast=True)
# 2. Continuous/numeric columns available before flight arrival
continuous_column_list = [
    "Actual_Elapsed_Time",
    "Scheduled_Departure_Hour",
    "Scheduled_Arrival_Hour",
    "Day_of_Week",
    "Day_of_Month",
    "Departure_Delay",
    "Distance",
    "Month",
]
# 3. Assemble and scale continuous features
continuous_assembler =
VectorAssembler(inputCols=continuous_column_list,
outputCol="continuousVector")
scaler = StandardScaler(inputCol="continuousVector",
outputCol="continuousScaled")
# 4. Final assembler to put everything into one "features" vector
```

```
feature_columns = categorical_encoded_list + ["continuousScaled"]
assembler = VectorAssembler(inputCols=feature_columns,
outputCol="features")


# In[20]:
# Test this much of the pipeline
# test_pipeline = Pipeline(stages=[indexer, encoder,
continuous_assembler, scaler, assembler])
# test_pipeline = Pipeline(stages=[indexer, encoder,
continuous_assembler, scaler])
# transformed_sdf = test_pipeline.fit(sdf).transform(sdf)
# transformed_sdf.show(n=2, vertical=True)
# TODO
# Optionally, save this transformed_sdf to your Trusted folder as it
will have all of the features.
# transformed_sdf.save.format(
trusted_filename =
f"{trusted_folder}/trusted_airline_data_data.parquet"
sdf.write.parquet(trusted_filename, mode='overwrite')

# In[21]:
# Create a LogisticRegression Estimator
lr = LogisticRegression()

# In[22]:
# Assemble all of the steps into a pipline
machine_learning_pipeline = Pipeline(stages=[indexer, encoder,
continuous_assembler, scaler, assembler, lr])

# In[23]:
# Split the sdf dataframe into training and test sets
trainingData, testData = sdf.randomSplit([0.7, 0.3])

# In[24]:
# Create a BinaryClassificationEvaluator to evaluate how well the
model works
evaluator = BinaryClassificationEvaluator(metricName="areaUnderROC")

# In[25]:
# Create a grid to hold hyperparameters
grid = ParamGridBuilder()
grid = grid.addGrid(lr.regParam, [0.0, 0.5, 1.0])
grid = grid.addGrid(lr.elasticNetParam, [0, 1])
# Build the parameter grid
grid = grid.build()
# Show the number of models to be tested
print('Number of models to be tested: ', len(grid))

# In[26]:
# Create the CrossValidator
```

```
# Note the 'estimator' is pointing to the complete
machine_learning_pipeline
cv = CrossValidator(estimator=machine_learning_pipeline,
               estimatorParamMaps=grid,
               evaluator=evaluator,
               numFolds=3)
# In[ ]:
# Use the CrossValidator to Fit the data
# This will train all of the models
all_models = cv.fit(trainingData)

# In[ ]:
# Choose the best model:
bestModel = all_models.bestModel

# In[ ]:
# Show the average metric on all of the model runs
print(f"Average metric: {all_models.avgMetrics}")

# In[ ]:
# Use the bestModel to run the model on the testData
test_results = bestModel.transform(testData)

# In[ ]:
# Evaluate the predictions
print(evaluator.evaluate(test_results))

# In[ ]:
# Show the label and predicted label
test_results.select(["Airline", "Origin", "Destination",
                     'Arrival_Delay', 'Distance','label',
'prediction']).show(truncate=False)

# In[ ]:
# Create the Confusion Matrix
cm =
test_results.groupby('label').pivot('prediction').count().fillna(0).so
rt('label', ascending=True).collect()

# In[ ]:
# Function to calculate recall and precision
def calculate_recall_precision(cm):
    tn = cm[0][1]
    fp = cm[0][2]
    fn = cm[1][1]
    tp = cm[1][2]
    precision = tp / ( tp + fp )
    recall = tp / ( tp + fn )
    accuracy = ( tp + tn ) / ( tp + tn + fp + fn )
    f1_score = 2 * ( ( precision * recall ) / ( precision + recall ) )
    return accuracy, precision, recall, f1_score
```

```
# In[ ]:
accuracy, precision, recall, f1_score =
calculate_recall_precision(cm)

# In[ ]:
print(accuracy, precision, recall, f1_score)

# In[ ]:
Cm

# In[ ]:
print("label,0.0,1.0\n", cm[0][0],",",cm[0][1],",",cm[0][2], "\n",
cm[1][0],",",cm[1][1],",",cm[1][2])

# In[ ]:
# Save the model to the models folder
model_path =
f"{models_folder}/airline_data_logistic_regression_model"
bestModel.write().overwrite().save(model_path)
```

Appendix e

Visualization

Code

```
# In[ ]:
#Distribution of Arrival Delays
arrival_delay_pd   =   sdf.select("Arrival_Delay").sample(False,   0.10,
seed=42).toPandas()
plt.figure(figsize=(10, 6))
sns.histplot(arrival_delay_pd["Arrival_Delay"], bins=50, kde=True)
plt.title("Distribution of Arrival Delays")
plt.xlabel("Arrival Delay (minutes)")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()

# In[32]:
#Flight Delay Rate by Airline
airline_delay_pd          =          sdf.sample(False,          0.10,
seed=42).groupBy("Airline").agg({"label": "avg"}) \
    .withColumnRenamed("avg(label)", "DelayRate").toPandas()
plt.figure(figsize=(12, 6))
sns.barplot(data=airline_delay_pd.sort_values("DelayRate",
ascending=False), x="Airline", y="DelayRate")
plt.title("Average Delay Rate by Airline")
plt.xlabel("Airline")
plt.ylabel("Delay Rate")
```

```
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# In[34]:
#Correlation Heatmap
numerical_cols = [
    "Actual_Elapsed_Time",                  "Scheduled_Departure_Hour",
"Scheduled_Arrival_Hour",
    "Day_of_Week",   "Day_of_Month",   "Departure_Delay",   "Distance",
"Month", "label"
]
correlation_data   =   sdf.select(numerical_cols).sample(False,   0.10,
seed=42).toPandas()
corr = correlation_data.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Matrix of Numerical Features")
plt.show()

# In[36]:
#ROC Curve
lr_model = bestModel.stages[-1]
roc_data = lr_model.summary.roc.toPandas()
plt.figure(figsize=(6, 6))
plt.plot(roc_data['FPR'], roc_data['TPR'], label="ROC Curve")
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label="Random
Guess")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.grid(True)
plt.show()

# In[50]:
lr_model = bestModel.stages[-1]

# In[51]:
attrs = []
metadata = test_results.schema["features"].metadata["ml_attr"]["attrs"]
for attr_type in ['binary', 'numeric']:
    attrs += metadata.get(attr_type, [])
feature_names = [attr['name'] for attr in sorted(attrs, key=lambda x:
x['idx'])]
# In[52]:
coefficients = lr_model.coefficients.toArray()
importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': coefficients,
    'AbsCoefficient': [abs(c) for c in coefficients]
}).sort_values(by='AbsCoefficient', ascending=False)
```

```
# In[53]:
plt.figure(figsize=(12, 6))
sns.barplot(data=importance_df.head(15),           x='AbsCoefficient',
y='Feature')
plt.title("Top 15 Most Influential Features in Delay Prediction")
plt.xlabel("Absolute Coefficient (Importance)")
plt.ylabel("Feature")
plt.tight_layout()
plt.grid(True)
plt.show()

# In[ ]:
#"The most influential features in predicting flight delays are
Departure_Delay, Scheduled_Departure_Hour,
#and certain airlines encoded in one-hot format. These features had the
highest absolute coefficients, indicating they strongly impact the
model's predictions.
#This aligns with expectations, as prior delays and flight timing are
key indicators of future delays."
```

Citations and Tools used

Apache Software Foundation. (2024). *MLlib: Machine learning library (Spark 3.5.0 documentation)*. https://spark.apache.org/docs/latest/ml-guide.html

Baruch College. (2025). *CIS 9760: Big Data Technologies – Course PDFs and Lecture Materials*. Department of Information Systems and Statistics, Zicklin School of Business, City University of New York.

Google Cloud. (2024). *Dataproc documentation*. https://cloud.google.com/dataproc

Google Cloud. (2024). *Cloud Storage documentation*. https://cloud.google.com/storage

Hunter, J. D. (2007). *Matplotlib: A 2D graphics environment*. *Computing in Science & Engineering*, 9(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Waskom, M. (2021). *Seaborn: Statistical data visualization*. Journal of Open Source Software, 6(60), 3021. https://doi.org/10.21105/joss.03021

Bulter22. (2022). *Airline data* [Data set]. Kaggle. https://www.kaggle.com/datasets/bulter22/airline-data